

095946- ADVANCED ALGORITHMS AND PARALLEL PROGRAMMING

Fabrizio Ferrandi

a.a. 2025-2026



- Randomized data structures
  - Random Treaps
  - Skip Lists



Dictionaries

- A dictionary is a collection of elements each of which has a unique search key
  - Uniqueness criteria may be relaxed (multiset)
  - (i.e. do not force uniqueness)
- Keep track of current members, with periodic insertions and deletions into the set
- Examples
  - Membership in a club, course records
  - Symbol table (contains duplicates)
  - Language dictionary (WordSmith, Webster, WordNet)
- Similar to database



Course Records

Dictionary

Member Record

key	student name	hw1	
123	Stan Smith	49	...
124	Sue Margolin	56	...
125	Billie King	34	...
	⋮		
167	Roy Miller	39	...

Satelite data



The dictionary problem

- **Given:** Universe $(U,<)$ of keys with a total order

- **Goal:** Maintain set $S \subseteq U$ under the following operations
- Search(x,S): Is $x \in S$?
- Insert(x,S): Insert x into S if not already in S.
- Delete(x,S): Delete x from S.



Extended set of operations

- Minimum(S): Return smallest key
- Maximum(S): Return largest key
- List(S): Output elements of S in increasing order by key
- Union($S1$,$S2$): Merge S_1 and S_2

  Condition: $\forall\ x_1 \in S_1\ ,\ x_2 \in S_2$: $x_1 < x_2$
- Split(S,x,S_1,S_2): Split S into S_1 and S_2.

  $\forall\ x_1 \in S_1\ ,\ x_2 \in S_2$: $x_1 \leq x$ and $x < x_2$



How to Implement a Dictionary?

- Sequences / Arrays
  - ordered
  - unordered
- Binary Search Trees
- AVL tree
- Splay tree

- Random Treaps
- Skip lists



Random Treaps

- treaps achieve essentially the same time bounds
- they do not require any explicit balance information
- the expected number of rotations performed is small for each operation
- simple to implement

- Skip lists similar benefits but alternative randomized data structure



Approach for randomized search trees

- If *n* elements are inserted in random order into a binary search tree, the expected depth is 1.39 log *n*

- Goal:
  - the search tree has the structure that would result if elements were inserted in the order of their priorities (property 1)

- Idea:
  - Each element *x* is assigned a priority chosen uniformly at random

$$\text{prio}(x) \in R$$



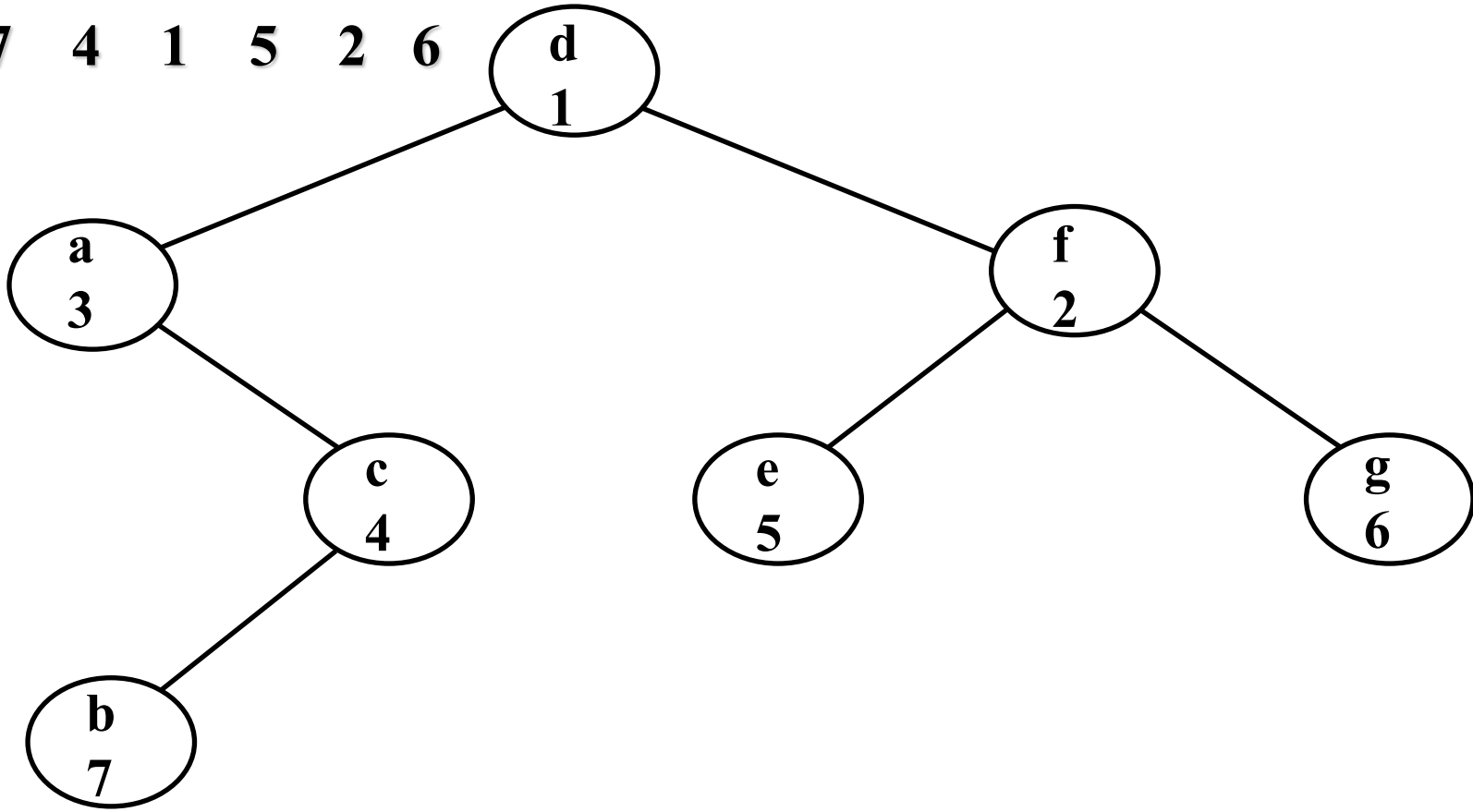
Treaps: tree + heap

- **Definition:** A treap is a binary tree
- Each node contains one element x with $\text{key}(x) \in U$ and $\text{prio}(x) \in R$
- The following properties hold:
  - Search tree property
    - For each element x:
    - elements y in the left subtree of x satisfy: $\text{key}(y) < \text{key}(x)$
    - elements y in the right subtree of x satisfy : $\text{key}(y) > \text{key}(x)$
  - Heap property
    - For all elements x,y:
      - If y is a child of x, then $\text{prio}(y) > \text{prio}(x)$.
      - All priorities are pairwise distinct.



Example

key	a	b	c	d	e	f	g
Priority	3	7	4	1	5	2	6





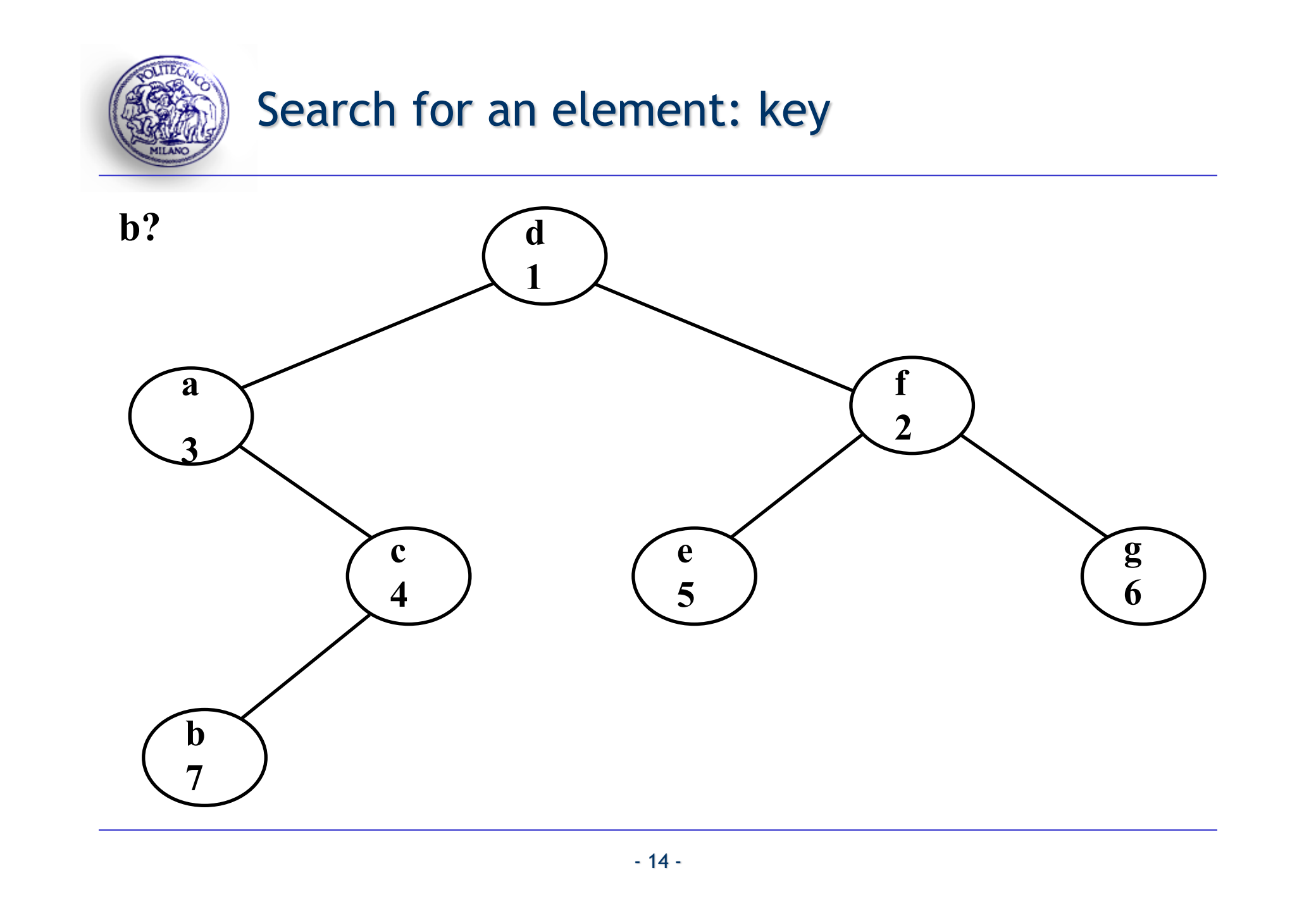
Treap uniqueness

- **Lemma:** For elements $x_1, \ldots, x_n$ with key(x_i) and prio(x_i), there exists a unique treap.



Treap structure

- the search tree has the structure that would result if elements were inserted in the order of their priorities


Search for an element: key
b?
d
1
a
3
f
2
c
4
e
5
g
6
b
7



Search for element with key *k*

```
1 v := root;
2 while v ≠ nil do
3    case key(v) = k : stop; "element found"
(successful search)
4       key(v) < k : v:= RightChild(v);
5       key(v) > k : v:= LeftChild(v);
6    endcase;
7 endwhile;
8 "element not found" (unsuccessful search)
```

Running time: O(# elements on the search path)



Analysis of the 'Search' operation

Let T be a treap with elements $x_1, \ldots, x_n$ x_i has i-th smallest key

n-th Harmonic number:

$$H_n = \sum_{k=1}^{n} 1/k$$

$$H_n = \ln n + O(1)$$

Lemma:

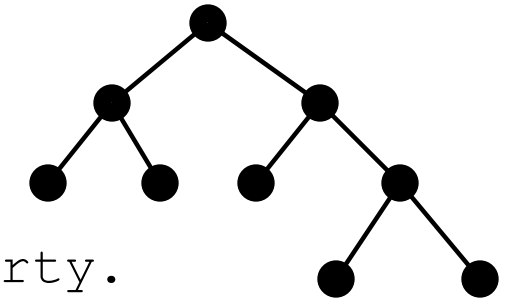
1. Successful search: The expected number of nodes on the path to x_m is $H_m + H_{n-m+1} - 1$.

2. Unsuccessful search : Let m be the number of keys that are smaller than the search key k. The expected number of nodes on the search path is $H_m + H_{n-m}$.



Inserting a new element *x*

1. Choose prio(x).
2. Search for the position of *x* in the tree.

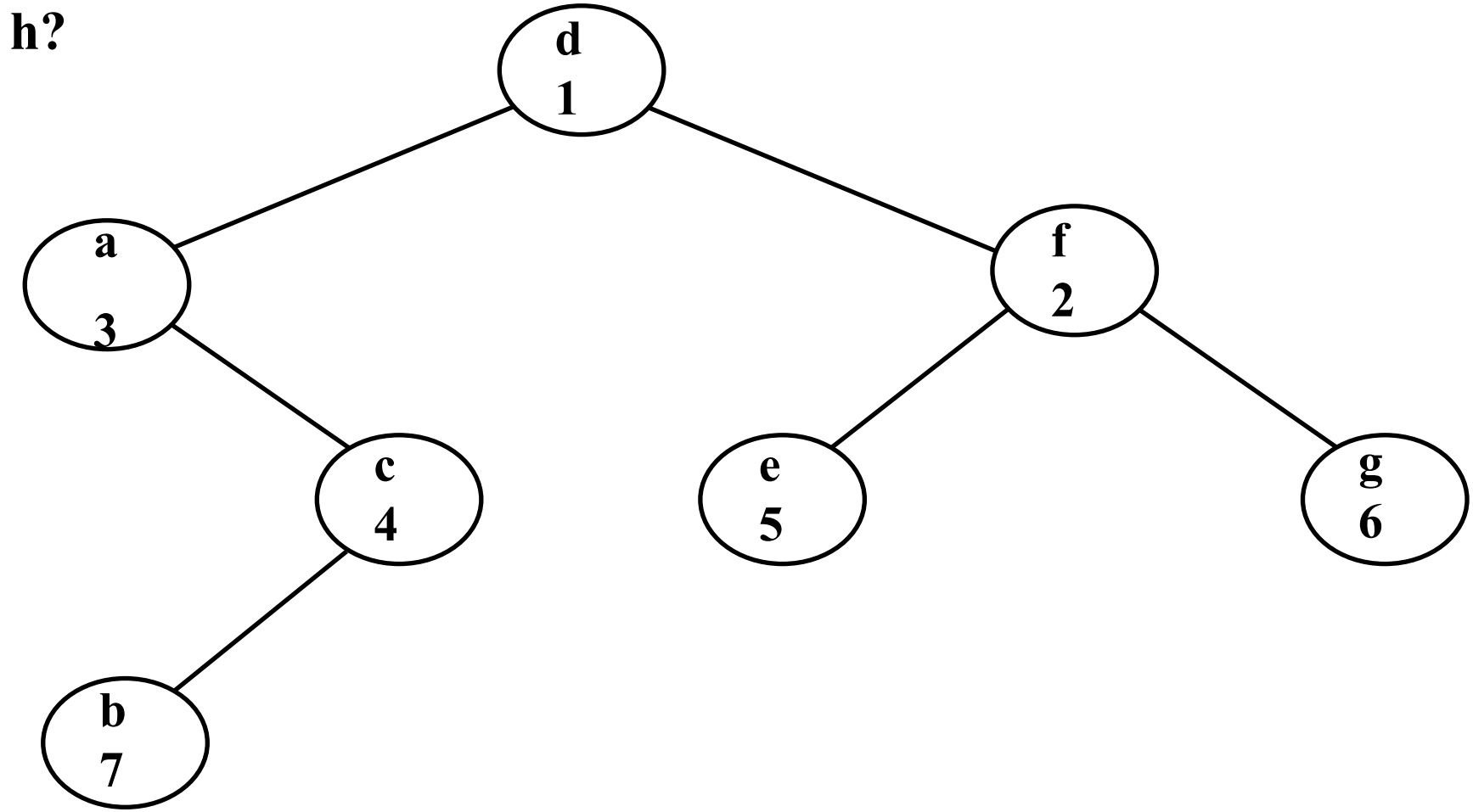


3. Insert *x* as a leaf.
4. Restore the heap property.

```
while prio(parent(x)) > prio(x) do
  if x is left child then RotateRight(parent(x))
  else RotateLeft(parent(x));
  endif
endwhile;
```



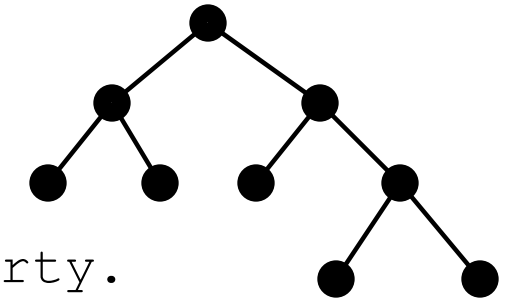
Insert an element





Inserting a new element *x*

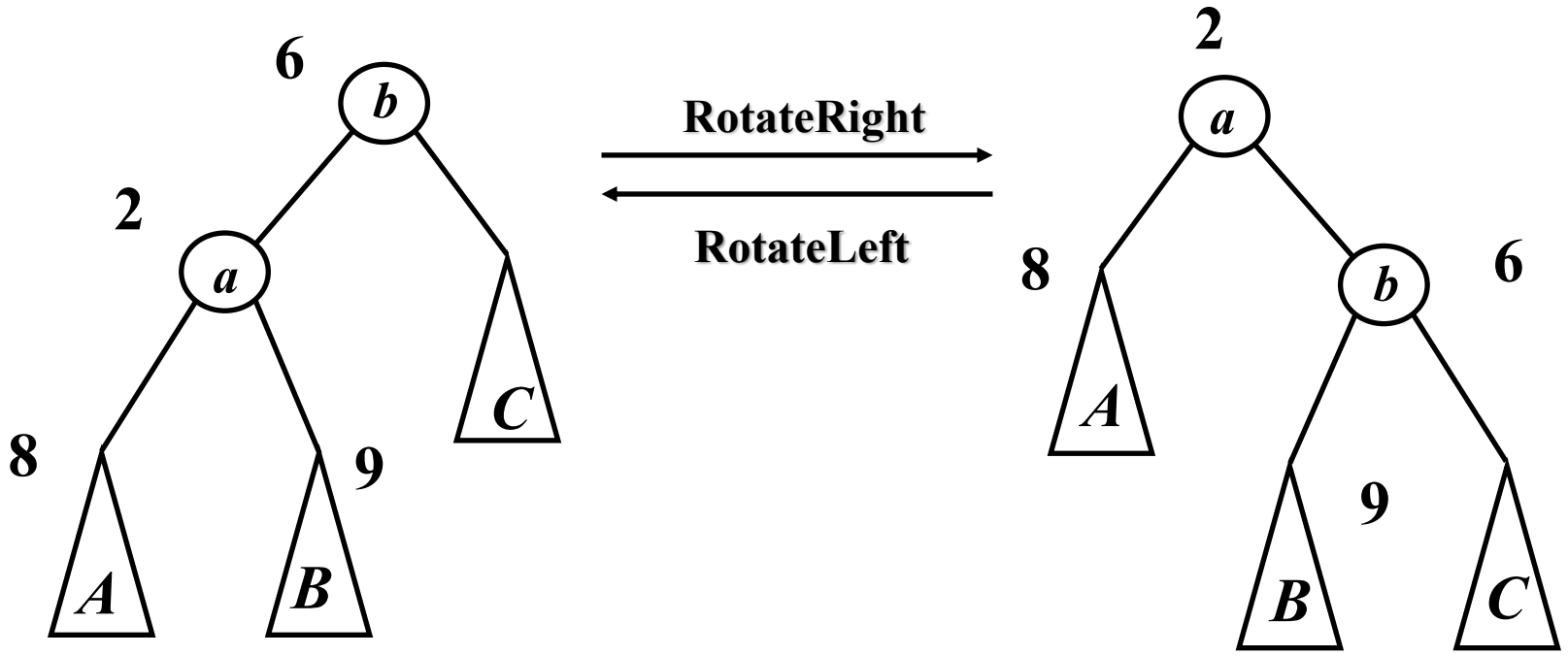
1. Choose prio(x).
2. Search for the position of *x* in the tree.



3. Insert *x* as a leaf.
4. Restore the heap property.

```
while prio(parent(x)) > prio(x) do
  if x is left child then RotateRight(parent(x))
  else RotateLeft(parent(x));
  endif
endwhile;
```

Rotations



- The rotations maintain the search tree property and restore the heap property.

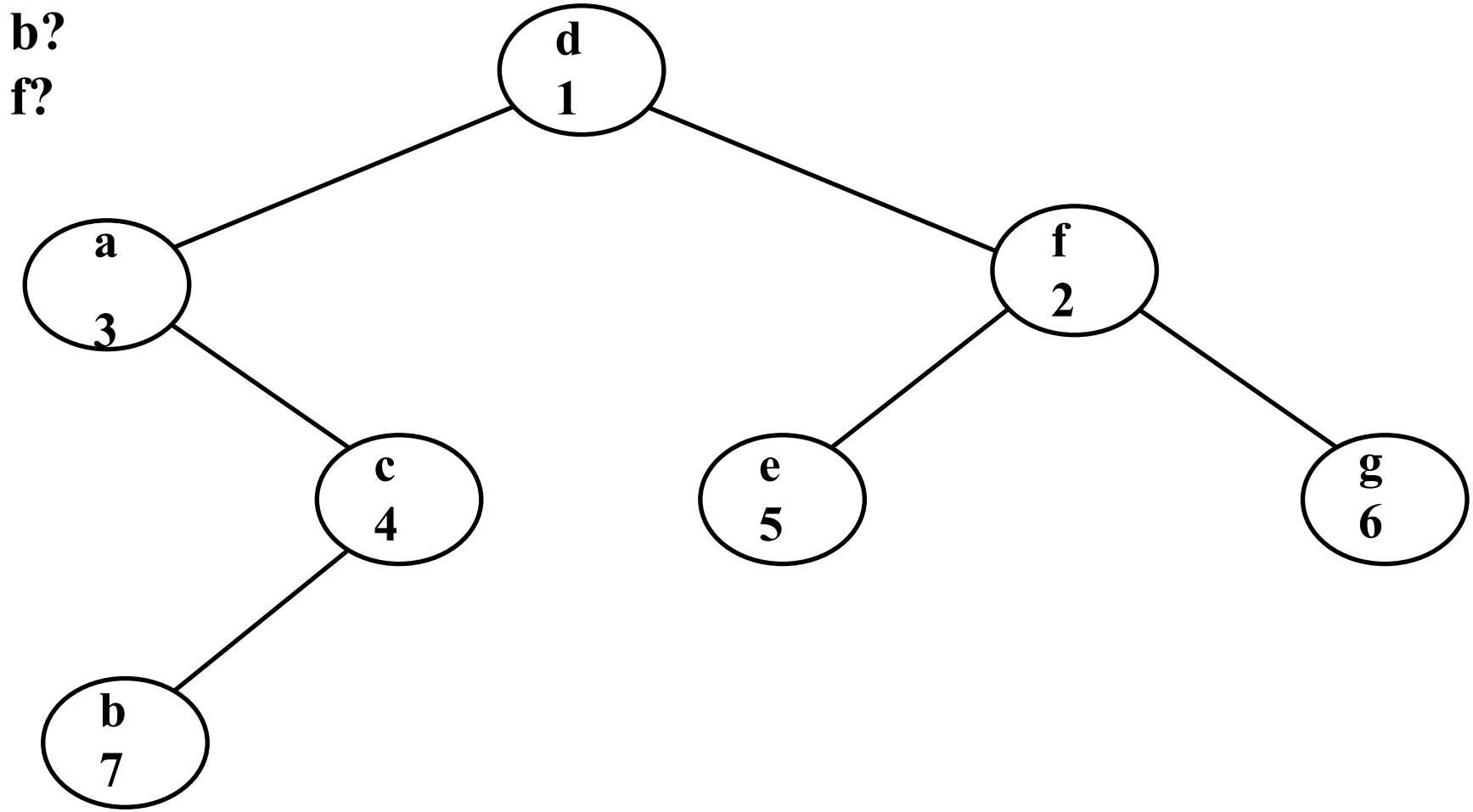


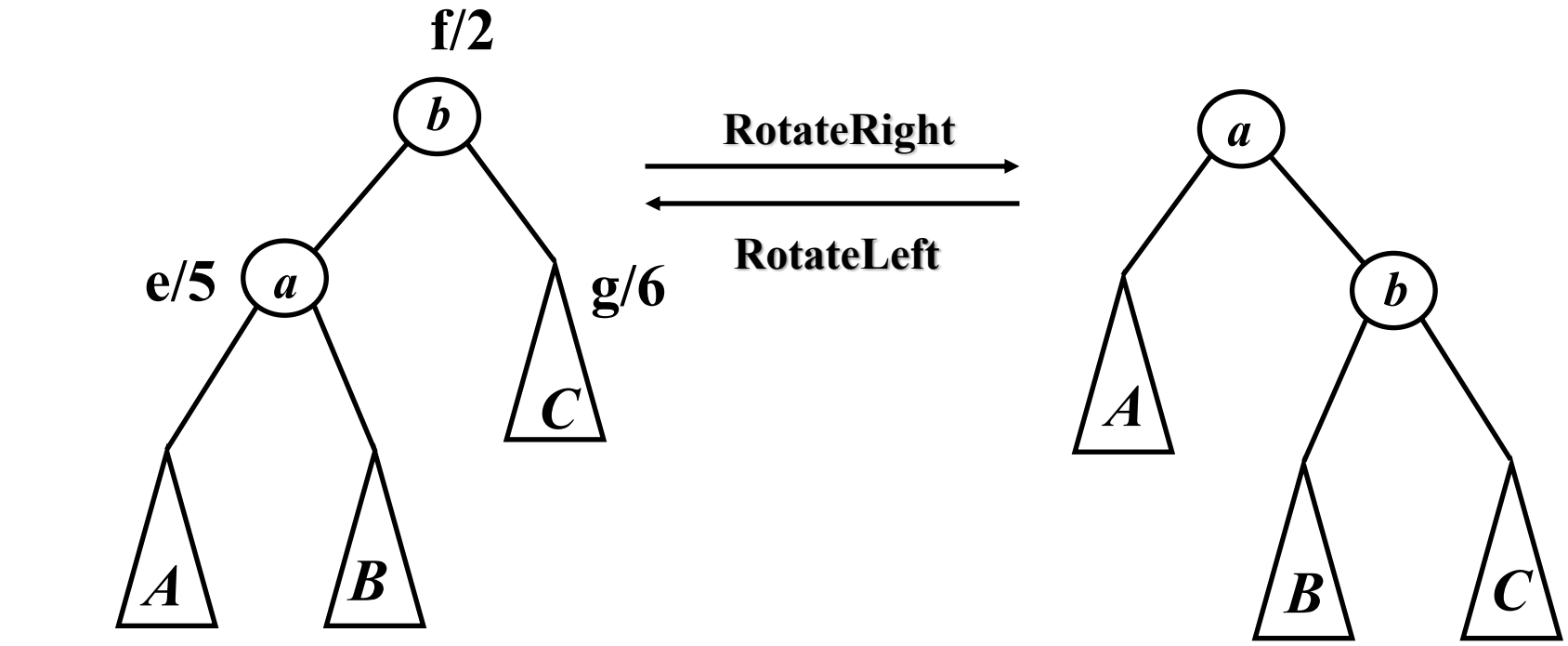
Deleting an element *x*

```
1.Find x in the tree.
2. while x is not a leaf do
       u := child with smaller priority;
       if u is left child then RotateRight(x))
       else RotateLeft(x);
       endif;
   endwhile;
3. Delete x;
```

Delete an element

b?
f?





Rotations



Analysis of ‘Insert’ and ‘Delete’ operations

Lemma: The expected running time of insert and delete operations is O(log *n*). The expected number of rotations is 2.

Proof: Analysis of insert (delete is the inverse operation)

rotations = depth of *x* after being inserted as a leaf (1)

- depth of *x* after the rotations (2)

Let $x = x_m$.

(2) Expected depth is $H_m + H_{n-m+1} - 1$.

(1) Expected depth is $H_{m-1} + H_{n-m} + 1$.

The tree contains *n-1* elements, *m-1* of them being smaller.

rotations = $H_{m-1} + H_{n-m} + 1 - (H_m + H_{n-m+1} - 1) < 2$



Extended set of operations

n = number of elements in treap T.

- Minimum(T): Return the smallest key. O(log n)
- Maximum(T): Return the largest key. O(log n)
- List(T): Output elements of S in increasing order. O(n)
- Union(T_1,T_2): Merge T_1 and T_2.

  Condition: $\forall\ x_1 \in T_1\ ,\ x_2 \in T_2$: key($x_1$) < key($x_2$)
- Split(T,k,T_1,T_2): Split T into T_1 and T_2.

  $\forall\ x_1 \in T_1\ ,\ x_2 \in T_2$: key($x_1$) ≤ k and k < key(x_2)



The ‘Split’ operation

Split(T,k,T_1,T_2): Split T into T_1 and T_2.

$\forall\ x_1 \in T_1\ ,\ x_2 \in T_2$: key($x_1$) ≤ k and k < key(x_2)

W.l.o.g. key k is not in T.

Otherwise delete the element with key k and re-insert it into T_1 after the split operation.

1. Generate a new element x with key(x)=k and prio(x)=-∞.
2. Insert x into T.
3. Delete the new root. The left subtree is T_1, the right subtree is T_2.



The ‘Union’ operation

Union(T_1,T_2): Merge T_1 and T_2.

Condition: $\forall\ x_1 \in T_1\ ,\ x_2 \in T_2$: key($x_1$) < key($x_2$)

1. Determine key k with key(x_1) < k < key(x_2)
   for all $x_1 \in T_1$and $x_2 \in T_2$.
2. Generate element x with key(x)=k and prio(x) = -∞.
3. Generate treap T with root x, left subtree T_1 and right subtree T_2.
4. Delete x from T.



Analysis

- **Lemma:** The expected running time of the operations Union and Split is O(log *n*).



Implementation

- Priorities from [0,1)

- Priorities are used only when two elements are compared to find out which of them has the higher priority.
- In case of equality, extend both priorities by bits chosen uniformly at random until two corresponding bits differ.

$p_1 = 0{,}010111001$
$p_2 = 0{,}010111001$

$p_1 = 0{,}010111001011$
$p_2 = 0{,}010111001010$



Skip lists

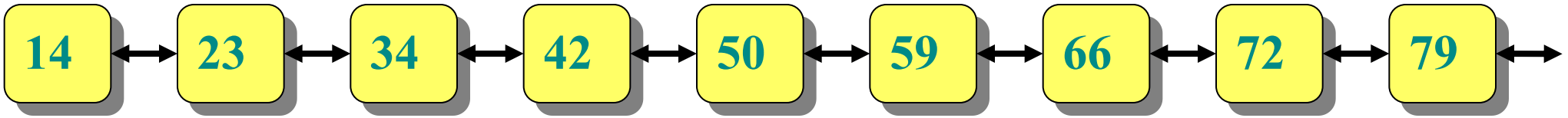
- Simple randomized dynamic search structure
  - Invented by William Pugh in 1989
  - Easy to implement
- Maintains a dynamic set of n elements in $O(\lg n)$ time per operation in expectation and *with high probability*
  - Strong guarantee on tail of distribution of $T(n)$
  - $O(\lg n)$ “almost always”



One linked list

Start from simplest data structure:
(sorted) linked list

- Searches take $\Theta(n)$ time in worst case
- How can we speed up searches?

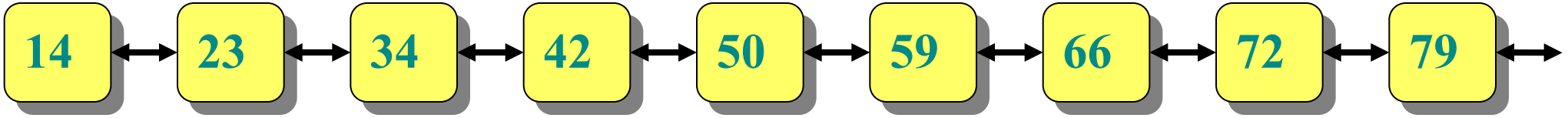




Two linked lists

Suppose we had *two* sorted linked lists
(on subsets of the elements)

- Each element can appear in one or both lists
- How can we speed up searches?

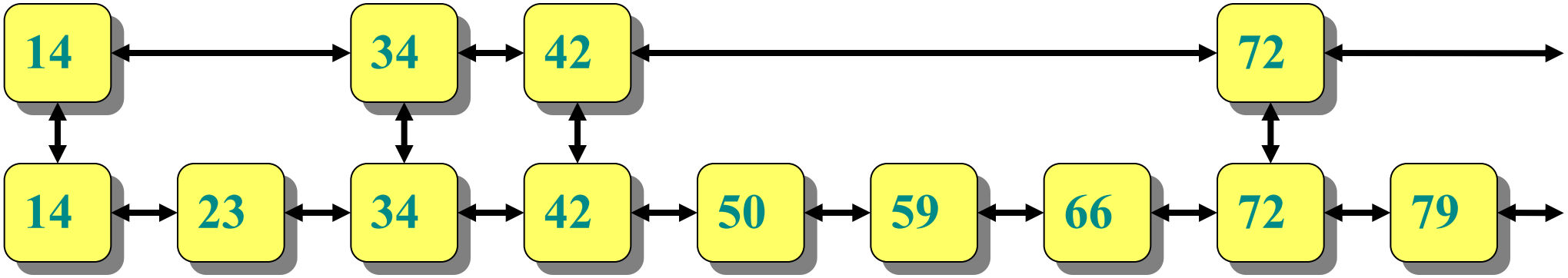




Two linked lists as a subway

IDEA: Express and local subway lines
(à la New York City 7th Avenue Line)

- Express line connects a few of the stations
- Local line connects all stations
- Links between lines at common stations

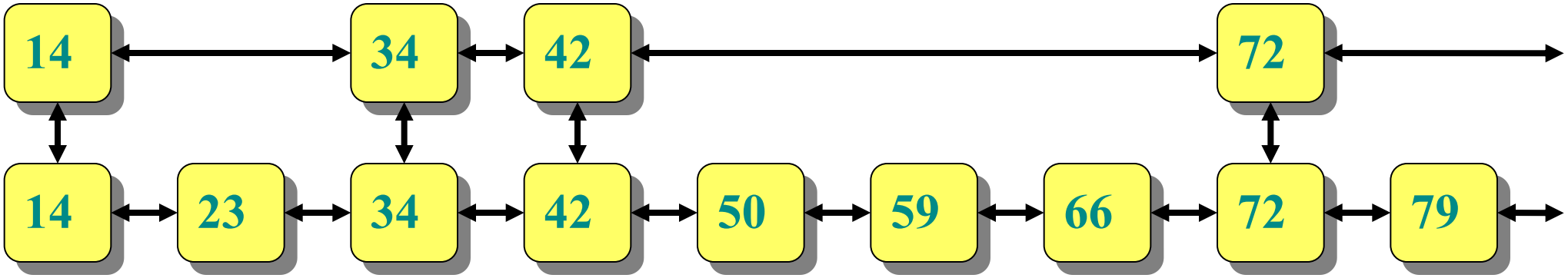




Searching in two linked lists

SEARCH(x):

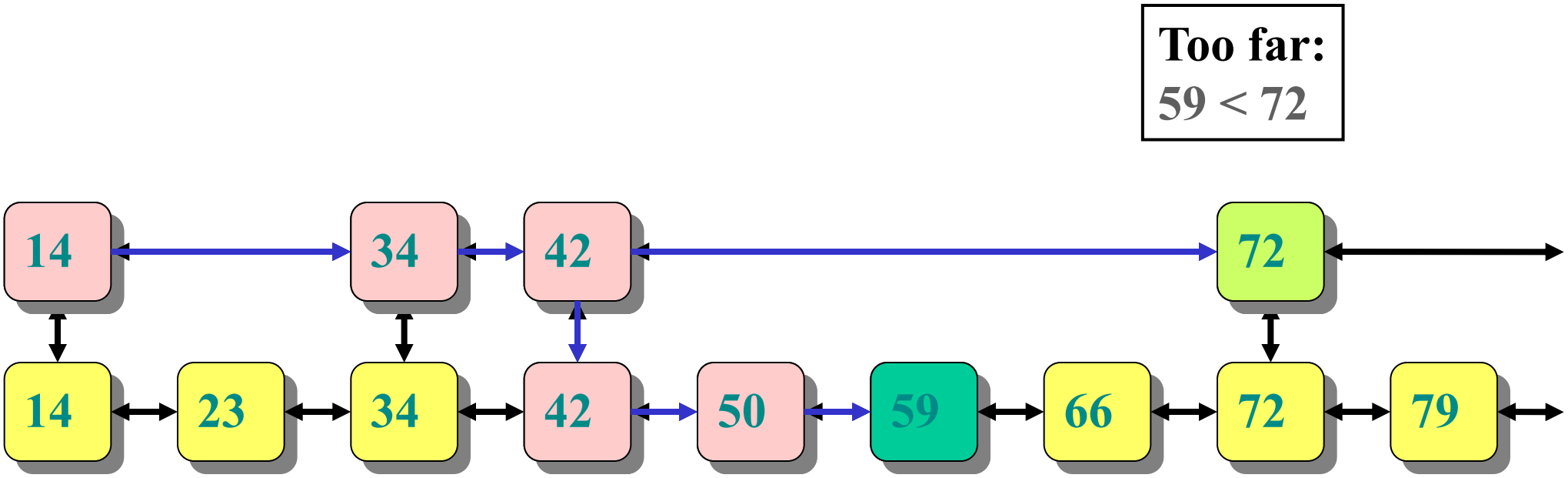
- Walk right in top linked list (L_1)
  until going right would go too far
- Walk down to bottom linked list (L_2)
- Walk right in L_2 until element found (or not)





Searching in two linked lists

EXAMPLE: SEARCH(59)

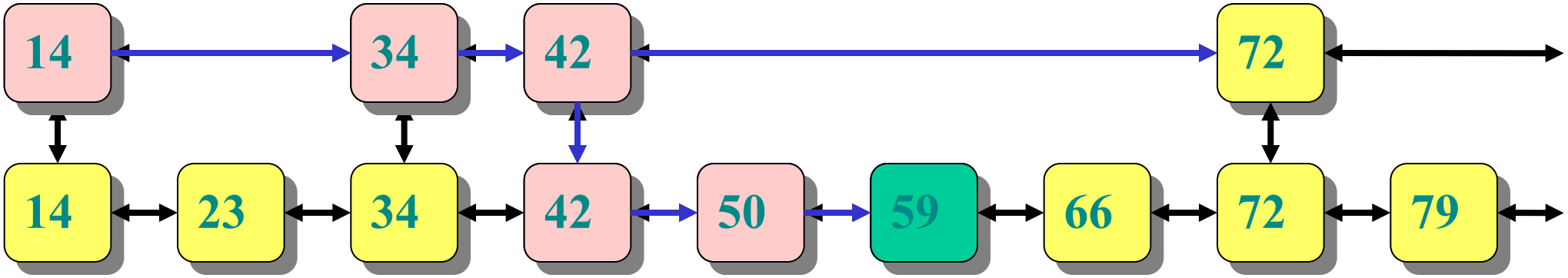




Design of two linked lists

QUESTION: Which nodes should be in L_1?

- In a subway, the "popular stations"
- Here we care about *worst-case performance*
- **Best approach:** Evenly space the nodes in L_1
- But *how many nodes* should be in L_1?



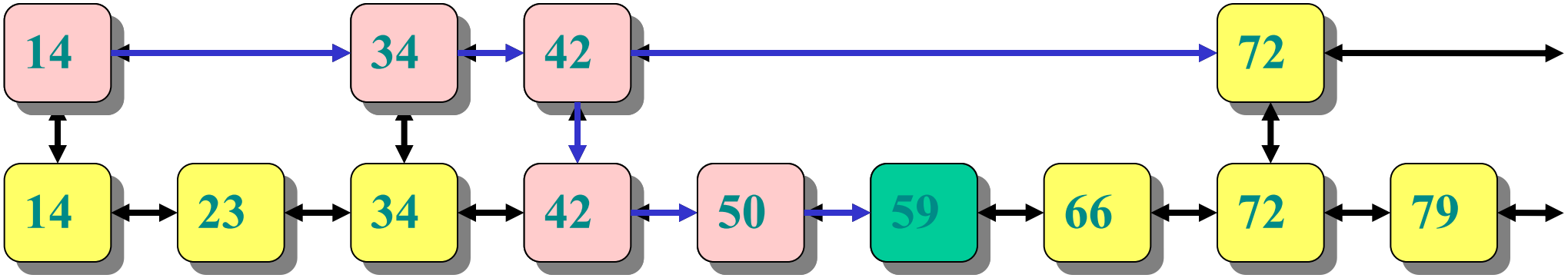


Analysis of two linked lists

ANALYSIS:

- Search cost is roughly $|L_1| + \frac{|L_2|}{|L_1|}$
- Minimized (up to constant factors) when terms are equal

$$|L_1|^2 = |L_2| = n \Rightarrow |L_1| = \sqrt{n}$$

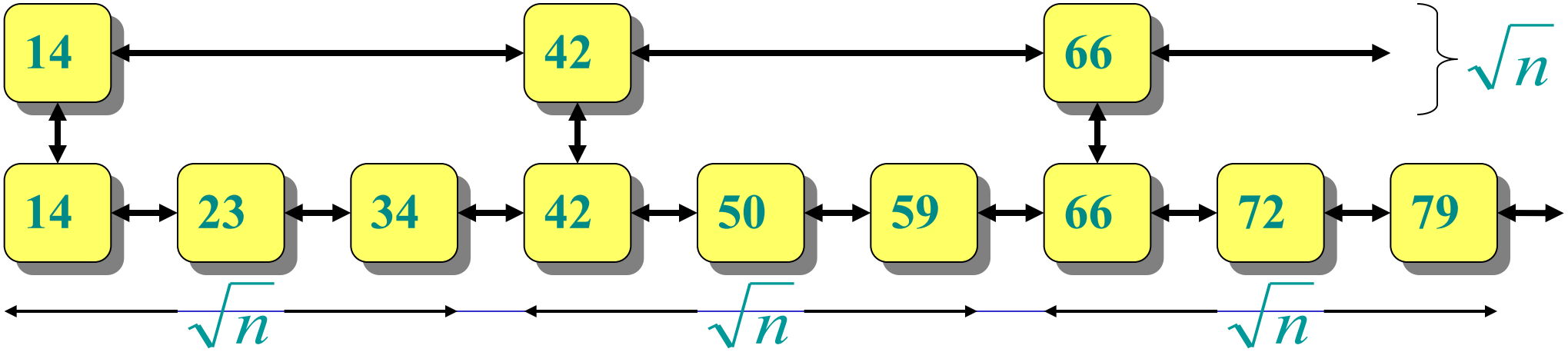


Analysis of two linked lists

ANALYSIS:

- $|L_1| = \sqrt{n}$ $\qquad$ $|L_2| = n$
- Search cost is roughly

$$|L_1| + \frac{|L_2|}{|L_1|} = \sqrt{n} + \frac{n}{\sqrt{n}} = 2\sqrt{n}$$

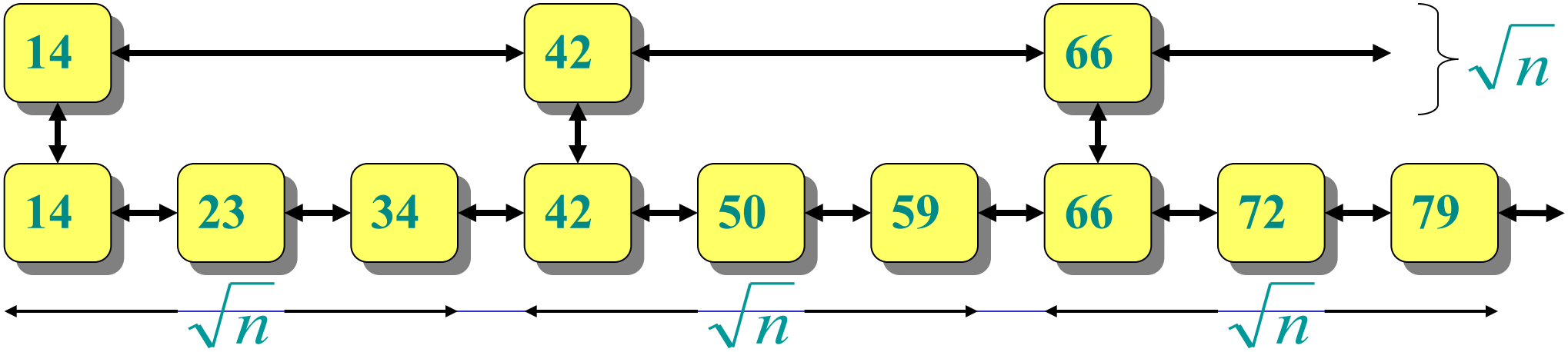




More linked lists

What if we had more sorted linked lists?

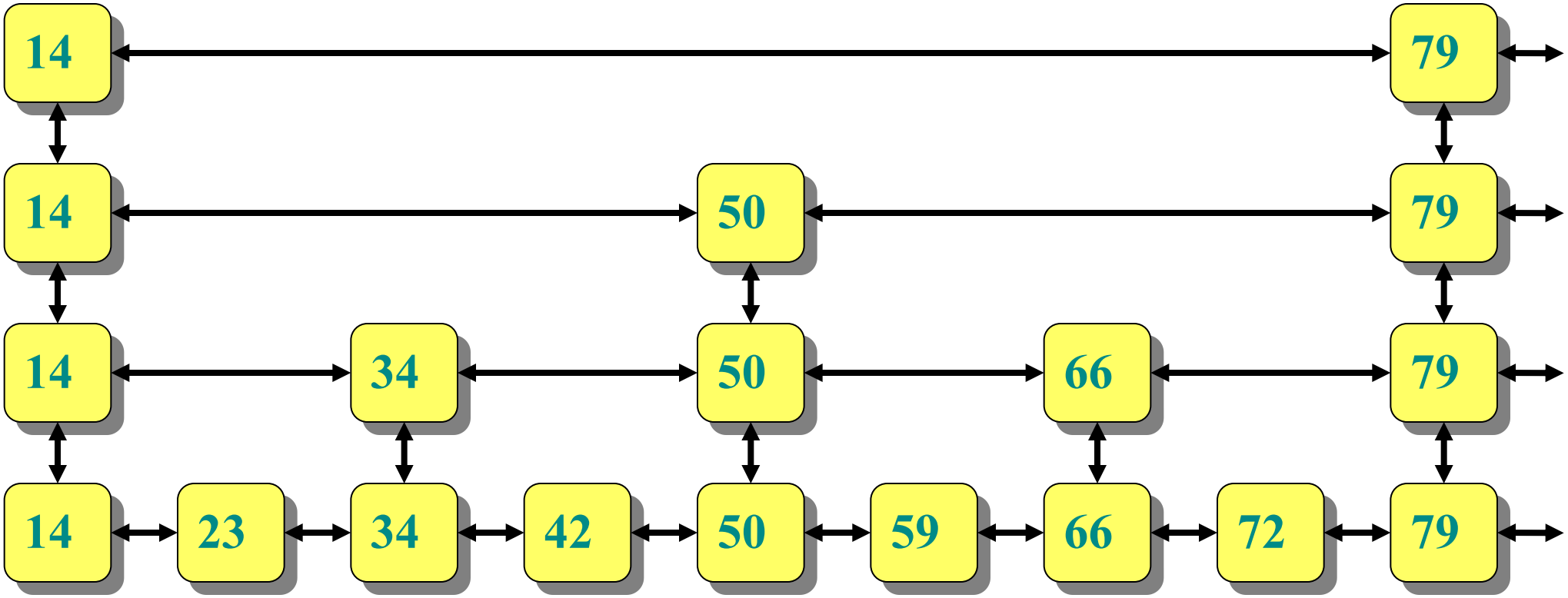
- 2 sorted lists $\Rightarrow 2 \cdot \sqrt{n}$
- 3 sorted lists $\Rightarrow 3 \cdot \sqrt[3]{n}$
- k sorted lists $\Rightarrow k \cdot \sqrt[k]{n}$
- $\lg n$ sorted lists $\Rightarrow \lg n \cdot \sqrt[\lg n]{n} = 2 \lg n$





lg *n* linked lists

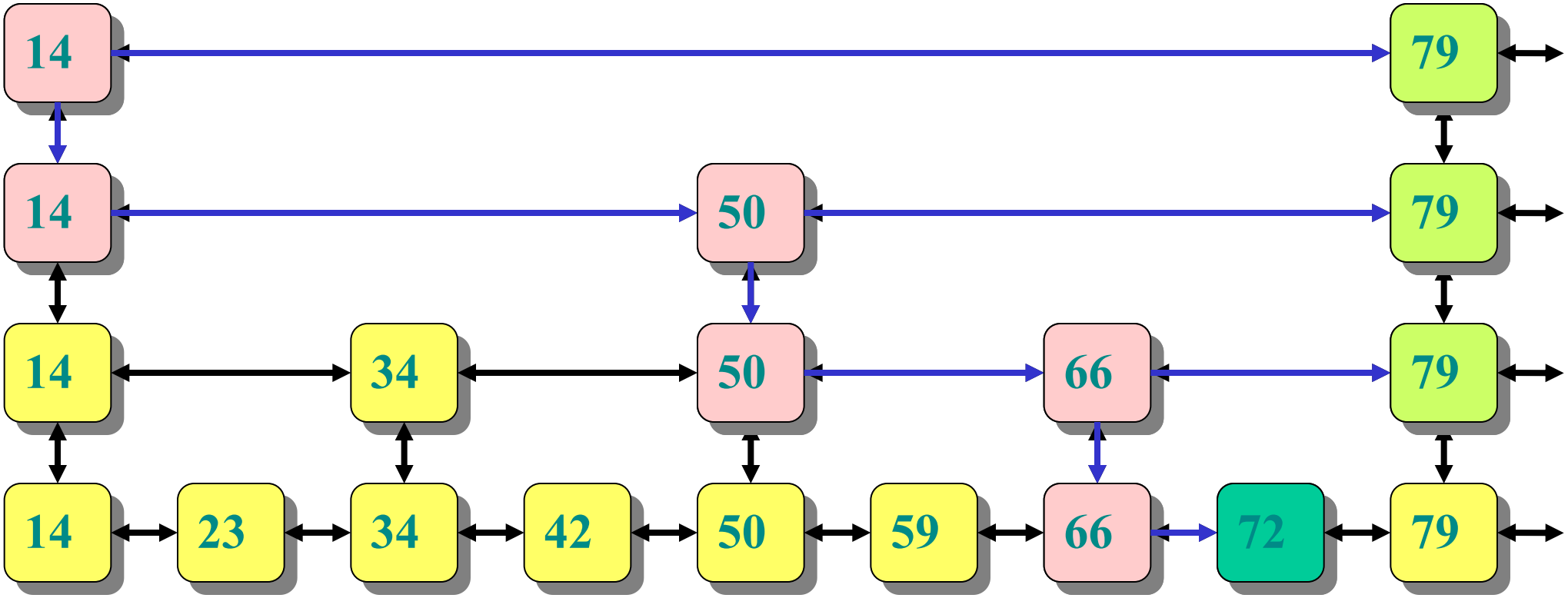
lg *n* sorted linked lists are like a binary tree





Searching in lg *n* linked lists

EXAMPLE: SEARCH(72)

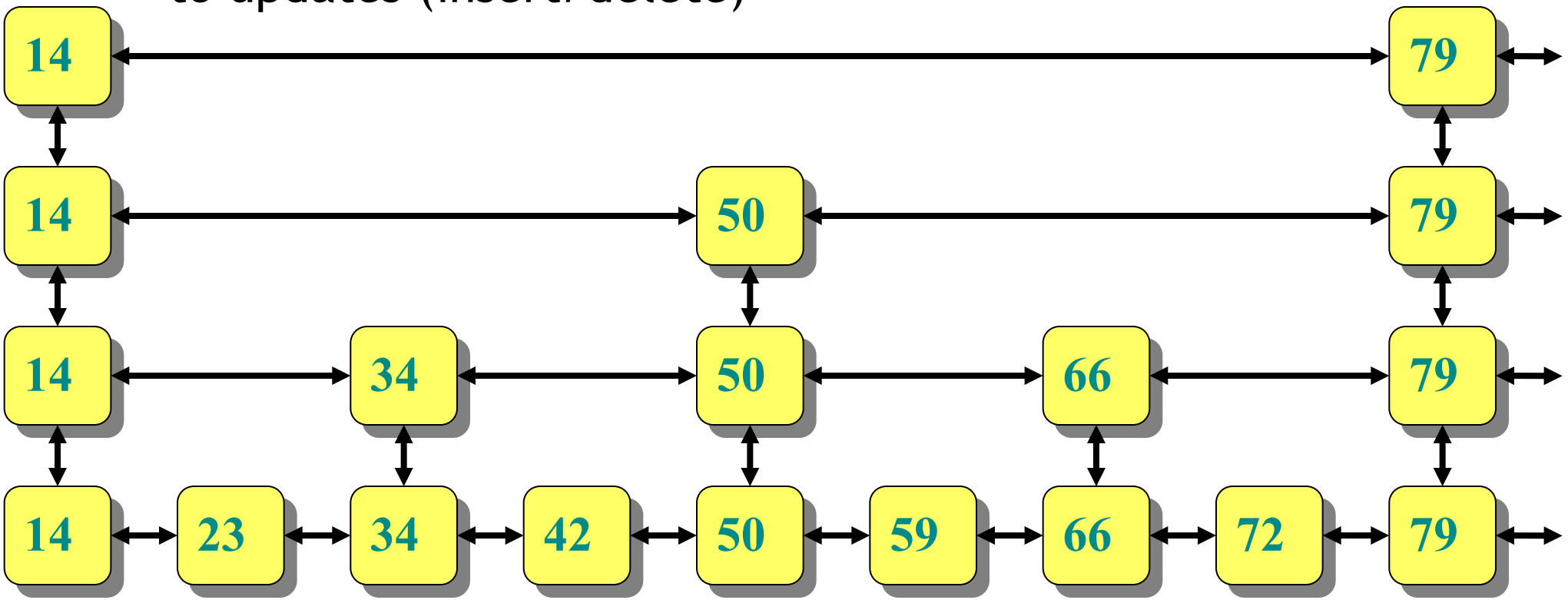




Skip lists

Ideal skip list is this lg *n* linked list structure

Skip list data structure maintains roughly this structure subject to updates (insert/delete)





INSERT(x)

To insert an element x into a skip list:

- SEARCH(x) to see where x fits in bottom list
- Always insert into bottom list

INVARIANT: Bottom list contains all elements

- Insert into some of the lists above…

QUESTION: To which other lists should we add x?

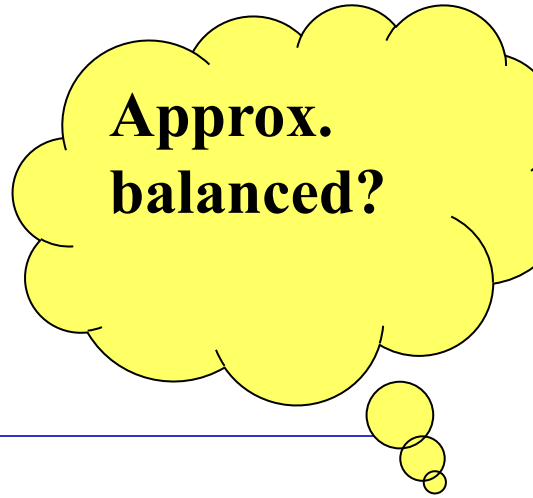


INSERT(*x*)

QUESTION: To which other lists should we add *x*?

IDEA: Flip a (fair) coin; if HEADS,
promote *x* to next level up and flip again

- Probability of promotion to next level = 1/2
- On average:
  - 1/2 of the elements promoted 0 levels
  - 1/4 of the elements promoted 1 level
  - 1/8 of the elements promoted 2 levels
  - etc.



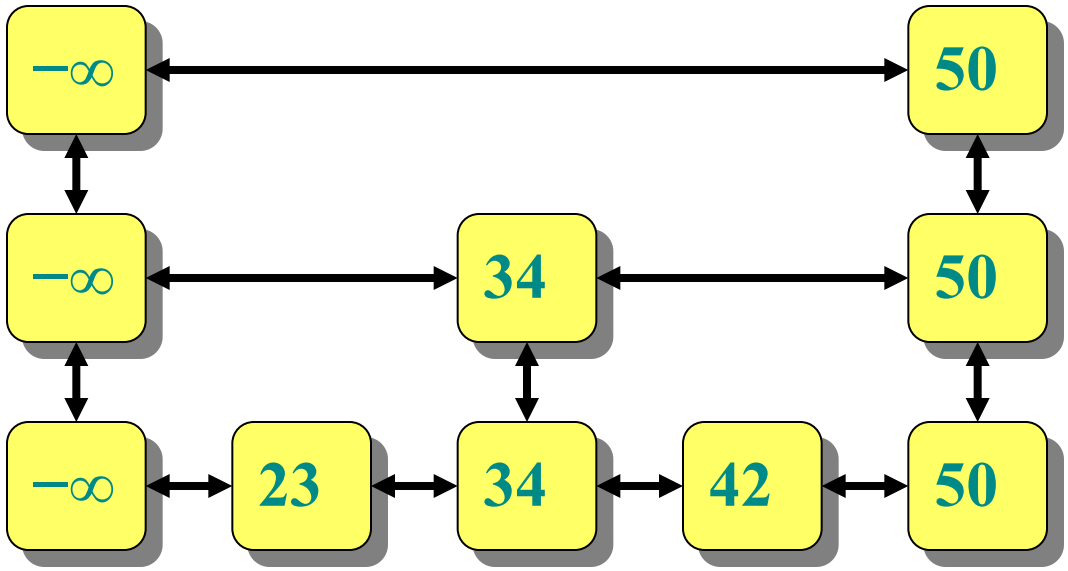


Example of skip list

EXERCISE: Try building a skip list from scratch by repeated insertion using a real coin

Small change:

- Add special $-\infty$ value to *every* list
  $\Rightarrow$ can search with the same algorithm





Skip lists

A ***skip list*** is the result of insertions (and deletions) from an initially empty structure (containing just $-\infty$)

- INSERT(x) uses random coin flips to decide promotion level
- DELETE(x) removes x from all lists containing it



Skip lists

A ***skip list*** is the result of insertions (and deletions) from an initially empty structure (containing just $-\infty$)

- INSERT(x) uses random coin flips to decide promotion level
- DELETE(x) removes x from all lists containing it

How good are skip lists? (speed/balance)

- **INTUITIVELY:** Pretty good on average
- **CLAIM:** Really, really good, almost always



With-high-probability theorem

THEOREM: *With high probability*, every search in an n-element skip list costs $O(\lg n)$



With-high-probability theorem

THEOREM: *With high probability*, every search in a skip list costs $O(\lg n)$

- **INFORMALLY:** Event E occurs ***with high probability*** (***w.h.p.***) if, for any $\alpha \geq 1$, there is an appropriate choice of constants for which
  E occurs with probability at least $1 - O(1/n^{\alpha})$
  - In fact, constant in $O(\lg n)$ depends on α
- **FORMALLY:** Parameterized event E_{α} occurs ***with high probability*** if, for any $\alpha \geq 1$, there is an appropriate choice of constants for which
  E_a occurs with probability at least $1 - c_{\alpha}/n^{\alpha}$



Acknowledge

- Based on Randomized Algorithms, by R. Motwani and P. Raghavan.
- Based on Lectures of Prof. Dr. Th. Ottmann and of Prof. Dr. Susanne Albers: http://electures.informatik.uni-freiburg.de/portal/web/guest
- Based on Lectures of Prof. Rada Mihalcea Univ. of North Texas